

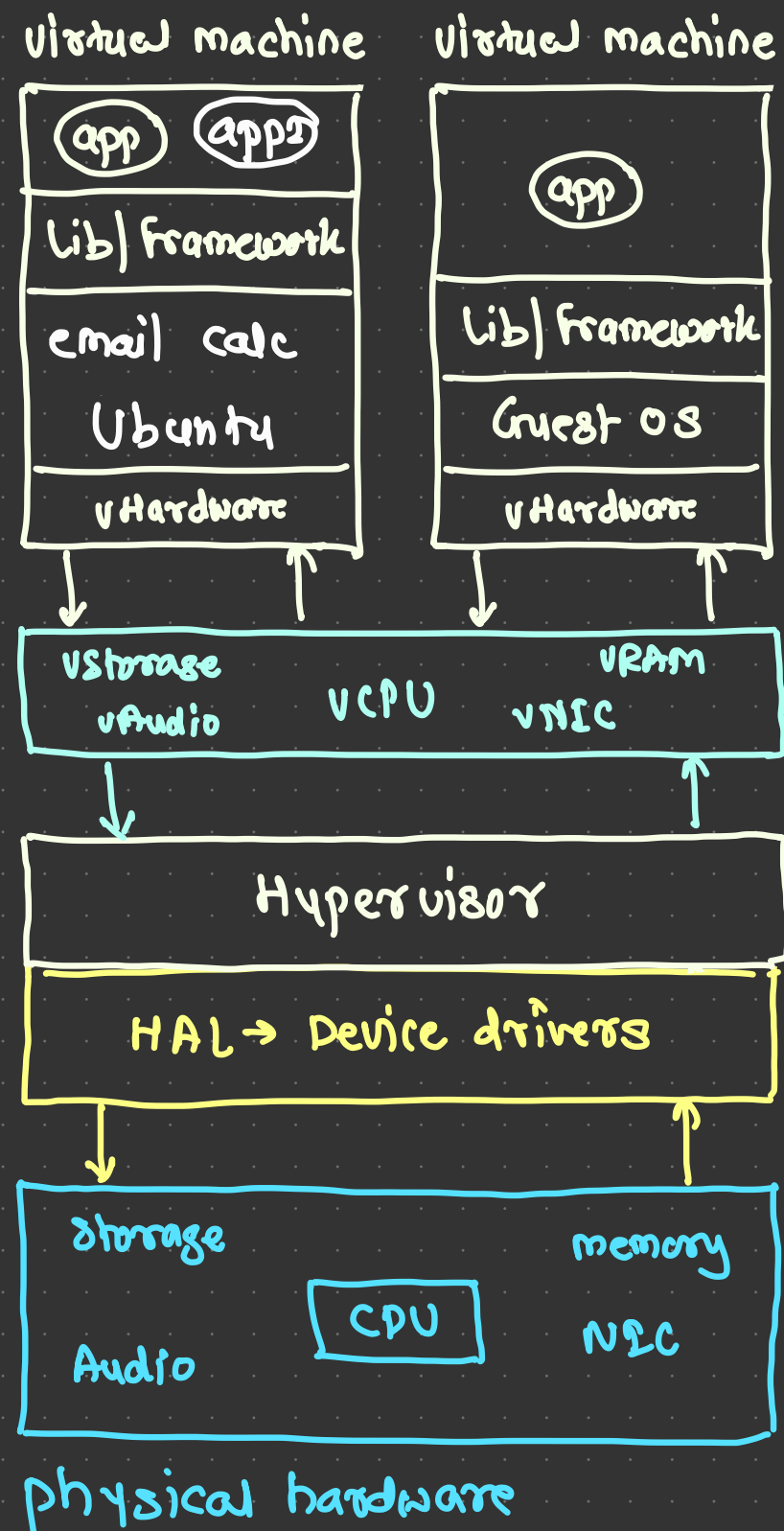


Containerization

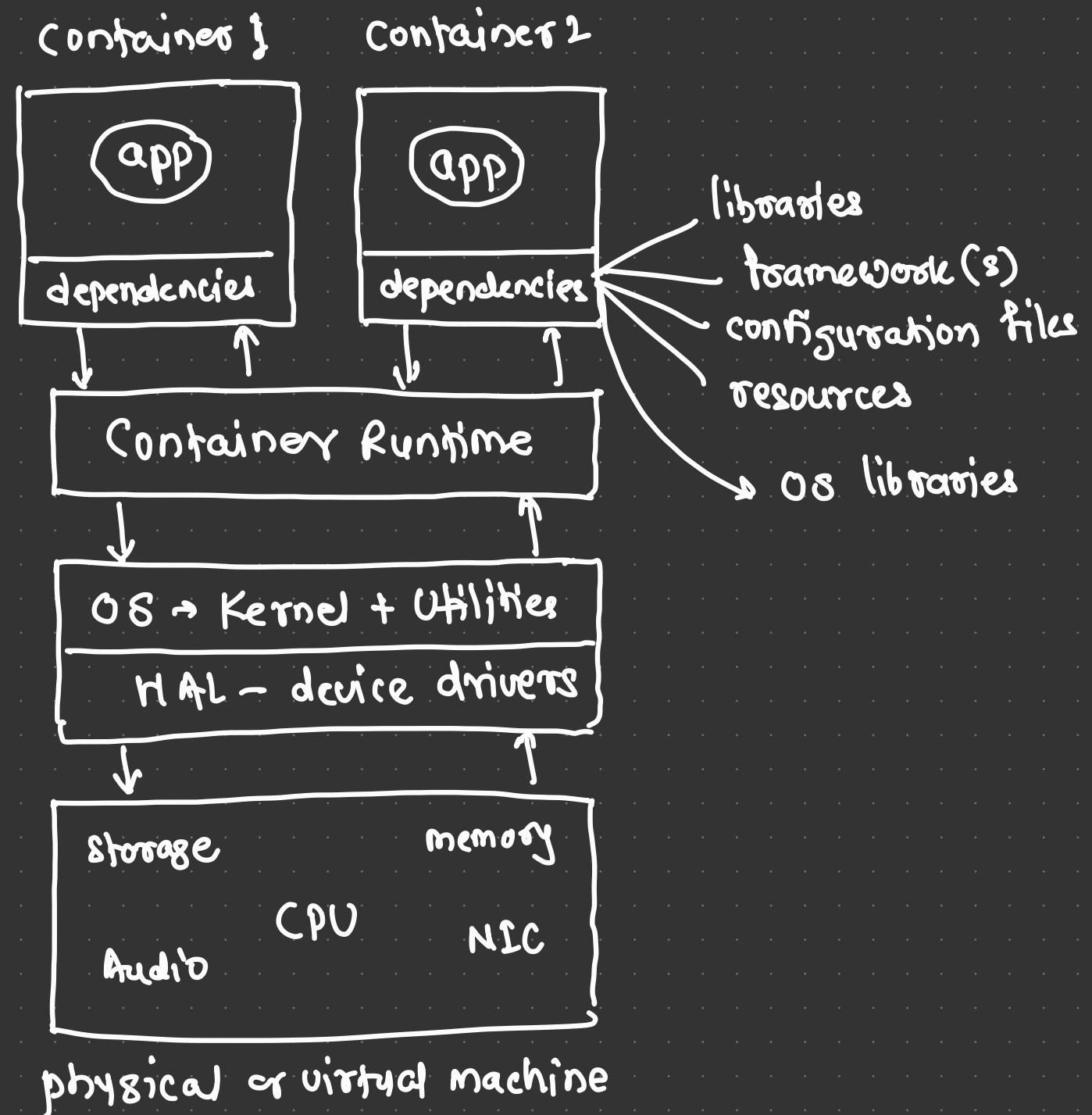
Operating System Virtualization

Hardware virtualization

Bare Metal → Type I



Operating System virtualization Sharing OS with the machine



latest version

dev env

python: 3.20

numpy: 2.1

pandas: 3.2

database: mysql 9

OS: windows



staging env

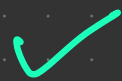
python: 3.20

numpy: 2.1

pandas: 3.2

database: mysql 9

OS: ubuntu 26.04



maximize uptime

↳ stable version

production environment

python: 3.12

numpy: 1.26

pandas: 3.0

database: mysql 8

OS: 24.04 LTS



What is Containerization

minimalistic OS

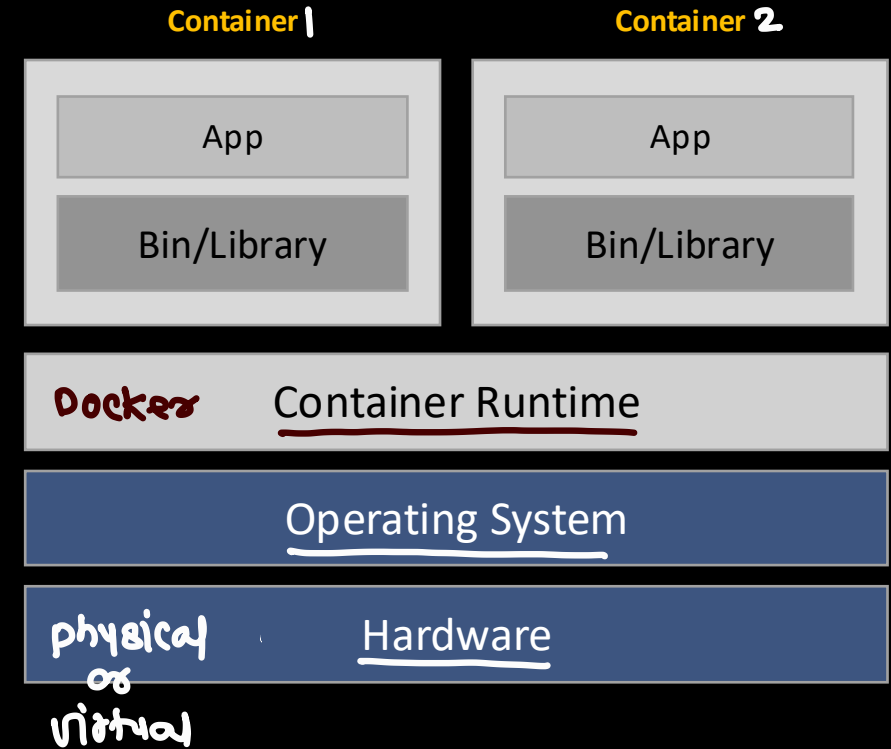
libs, frameworks,
resources, config



- Containerization is the packaging of software code with just the operating system (OS) libraries and dependencies required to run the code to create a single lightweight executable—called a **container**—that runs consistently on any infrastructure
- More portable and resource-efficient than virtual machines (VMs), containers have become the de facto compute units of modern cloud-native applications
- Containerization allows developers to create and deploy applications faster and more securely
- With traditional methods, code is developed in a specific computing environment which, when transferred to a new location, often results in bugs and errors
- For example, when a developer transfers code from a desktop computer to a VM or from a Linux to a Windows operating system
- Containerization eliminates this problem by bundling the application code together with the related configuration files, libraries, and dependencies required for it to run
- This single package of software or “container” is abstracted away from the host operating system, and hence, it stands alone and becomes portable—able to run across any platform or cloud, free of issues

Container deployment

- Containers are similar to VMs, but they have relaxed isolation properties to share the Operating System (OS) among the applications
- Therefore, containers are considered lightweight
- Similar to a VM, a container has its own filesystem, CPU, memory, process space, and more
- As they are decoupled from the underlying infrastructure, they are portable across clouds and OS distributions



Containerization vs Virtualization



Virtual Machine	Container
<u>Hardware level virtualization</u> → shared hardware	<u>OS virtualization</u> → shared OS
<u>Heavyweight (bigger in size)</u> → has guest OS	<u>Lightweight (smaller in size)</u> → No guest OS
<u>Slow provisioning</u> → OS requires booting	<u>Real-time and fast provisioning</u> → No OS booting
<u>Limited Performance</u>	<u>Native performance</u>
<u>Fully isolated</u>	<u>Process-level isolation</u>
<u>More secure</u>	<u>Less secure</u>
<u>Each VM has separate OS</u>	<u>Each container can share OS resources</u>
<u>Boots in minutes</u>	<u>Boots in seconds ready in seconds</u>
<u>Pre-configured VMs are difficult to find and manage</u>	<u>Pre-built containers are readily available</u>
<u>Can be easily moved to new OS</u> → mutable	<u>Containers are destroyed and recreated</u> → immutable
<u>Creating VM takes longer time</u>	<u>Containers can be created in seconds</u>

Benefits

↪ Container Runtime

- Portability - Run anywhere Docker is installed
- Efficiency - Lightweight, uses fewer resources than VMs
- Consistency - Same environment from dev to production
- Scalability - Easy to run multiple instances (horizontal scaling)
- Version Control - Track changes to environments
- Isolation - Secure separation between applications — process based isolation





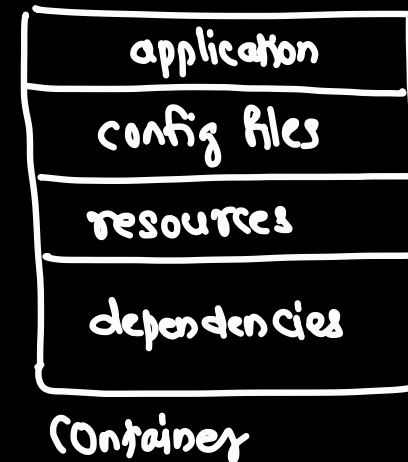
Docker



What is Docker ? → container Runtime



- Docker is an open-source platform that automates the deployment, scaling, and management of applications inside lightweight, portable containers
- It allows you to package an application with all its dependencies into a standardized unit called a container
- **The Core Problem Docker Solves**
 - **Without Docker (Traditional Approach)**
 - "It works on my machine, but not on the server"
 - Conflicts between different applications sharing the same system
 - Complicated setup and dependency management
 - Long deployment times
 - **With Docker**
 - Runs consistently anywhere (developer laptop, test server, production)
 - Applications are isolated from each other ✖
 - Simple, repeatable setup
 - Fast, reliable deployments



Little history about Docker



- Docker Inc, started by Solomon Hykes, is behind the docker tool
- Docker Inc started as PasS provider called as dotCloud
- In 2013, the dotCloud became Docker Inc
- Docker Inc was using Linux Containers (LXC) before version 0.9
- After 0.9 (2014), Docker replaced LXC with its own library libcontainer which is developed in Go programming language
- Its not the only solution for containerization
 - "FreeBSD Jails", launched in 2000
 - LXD is next generation system container manager build on top of LXC and REST APIs
 - Google has its own open source container technology Imctfy (Let Me Contain That For You)
 - Rkt is another option for running containers

Docker Architecture → client - server architecture uses REST APIs

client → dockers
server → dockerd

■ Docker Client

- Primary interface for users to interact with Docker
- Commands like docker run, docker build, docker pull
- Can communicate with multiple daemons
- Uses REST API over UNIX socket or network

■ Docker Daemon (dockerd)

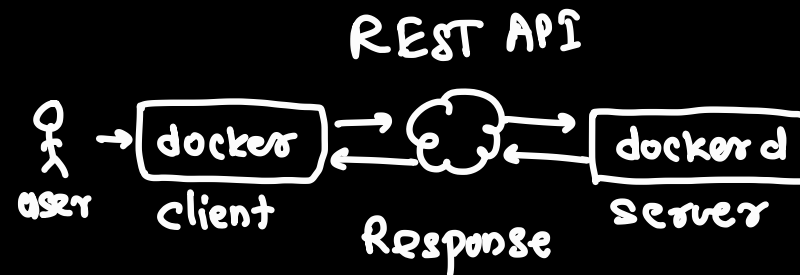
- Background service that manages Docker objects
- Listens for Docker API requests
- Manages containers, images, networks, volumes
- Can communicate with other daemons (Swarm mode) *
- Responsibilities:
 - Create and manage containers
 - Build and store images
 - Manage networks and volumes
 - Handle resource isolation

■ Docker Registry → collection of images

- Stores Docker images
- Public: Docker Hub (default)
- Private: Self-hosted (e.g., Amazon ECR, Google Container Registry)
- docker pull: downloads images
- docker push: uploads images

■ Docker Objects (Managed by Daemon)

- Images: Read-only templates
- Containers: Runnable instances
- Networks: Communication channels
- Volumes: Persistent storage



Modes of Operation

■ Default (Linux Containers)

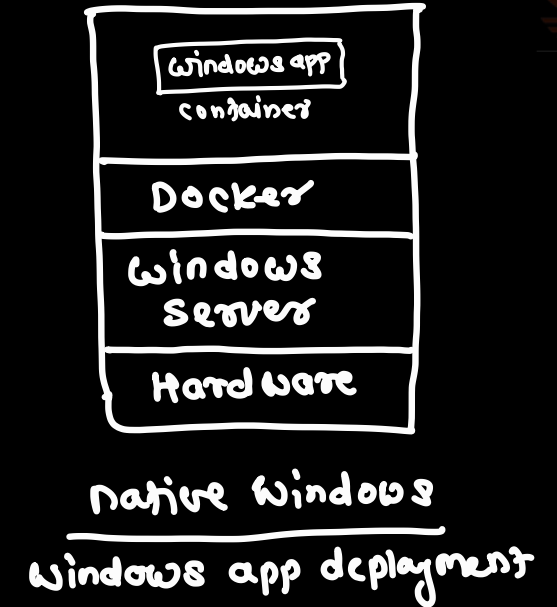
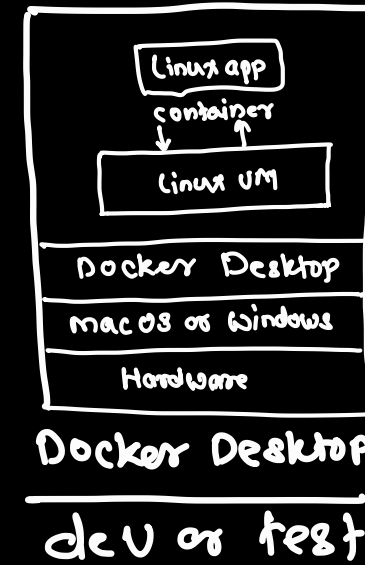
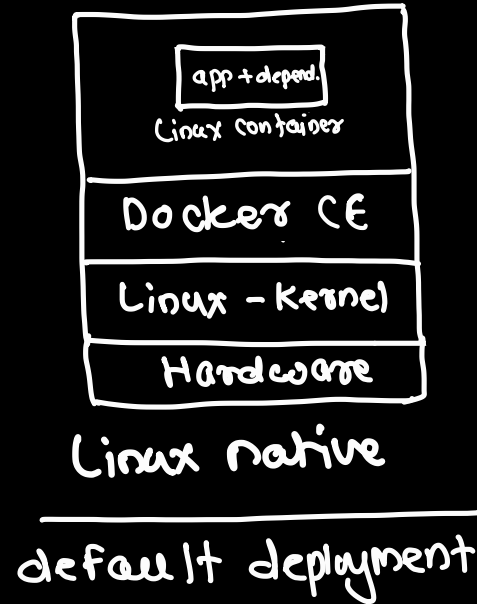
- Native performance
- Uses Linux kernel features
- Most common deployment

■ Docker Desktop (Mac/Windows)

- Runs Linux VM (HyperKit/Hyper-V)
- Linux containers run inside VM
- Provides seamless integration

■ Native Windows Containers

- Windows Server containers
- Uses Windows isolation
- Limited to Windows images



Docker Objects (Entity)

Container → immutable + stateless + cannot persist data

- Images: read only template with instructions for creating docker containers
- Container: running instance of a docker image (app + dependencies)
- Network: network interface used to connect the containers to each other or external networks
- Volumes: used to persist the data generated by and used by the containers
- Registry: private or public collection of docker images
- Service: used to deploy application in a docker multi node cluster (swarm) → orchestration
- Secrets: used to store secretes like API Keys, Passwords
- Nodes: physical or virtual machines running docker engine (swarm)
- Stacks: group of related services (swarm)



Docker Images



What is Docker image ?

- A **Docker image** is a **read-only template** that contains everything needed to run an application — including the **code, runtime, libraries, environment variables, and configuration files**
- Images are the **blueprint** for creating Docker containers
- An image acts like a snapshot of a filesystem and its configuration
- When you run an image, Docker creates a container based on that image
- You can build your own images or pull prebuilt ones from registries (like Docker Hub)
- Docker images are built using a **layered file system**
 - Base Layer: The foundation — e.g., ubuntu, alpine, node
 - Intermediate Layers: Contain instructions like installing dependencies or copying code
 - Top Layer: The final application code and entry point

```
# Base image
FROM python:3.11

# Set working directory
WORKDIR /app

# Copy source code
COPY . .

# Install dependencies
RUN pip install -r requirements.txt

# Command to run
CMD ["python", "app.py"]
```



Docker image commands

- **docker build:** Build a new image from a Dockerfile
- **docker image ls:** List all images on the system
- **docker pull:** Download an image from a registry
- **docker push:** Upload an image to a registry
- **docker image rm:** Remove an image
- **docker history:** Show layers of an image
- **docker image inspect:** View image metadata

Advantages



- **Portability:** Runs consistently anywhere Docker is supported
- **Reusability:** Same base image can be used for multiple apps
- **Efficiency:** Layered architecture saves space
- **Versioning:** Tags (:v1, :v2) track image versions
- **Automation:** Integrates with CI/CD pipelines easily



Docker Container

Docker Container



- It is a running aspect of docker image
- Contains one or more running processes
- It is a self-contained environment
- It wraps up an application into its own isolated box (application running inside a container has no knowledge of any other applications or processes that exist outside the container)
- A container can not modify the image from which it is created
- It consists of
 - Your application code
 - Dependencies
 - Networking
 - Volumes
- Containers are stored under `/var/lib/docker`
- This directory contains images, containers, network volumes etc

Basic Operations



- Creating container
- Starting container
- Running container
- Listing running containers
- Listing all containers
- Getting information of a container
- Stopping container
- Deleting container

Commands



Command	Description	Example
<code>docker container run</code>	Create and start a container	<code>docker run -d nginx</code>
<code>docker container ls</code>	List running containers	<code>docker ps</code>
<code>docker container ls -a</code>	List all containers (including stopped)	<code>docker ps -a</code>
<code>docker container stop</code>	Stop one or more running containers	<code>docker stop myapp</code>
<code>docker container start</code>	Start an existing container	<code>docker start myapp</code>
<code>docker container restart</code>	Restart a container	<code>docker restart myapp</code>
<code>docker container rm</code>	Remove a stopped container	<code>docker rm myapp</code>
<code>docker container logs</code>	View container logs	<code>docker logs myapp</code>
<code>docker container exec</code>	Run a command inside a running container	<code>docker exec -it myapp bash</code>
<code>docker container inspect</code>	View detailed information about a container	<code>docker inspect myapp</code>
<code>docker container stats</code>	Display resource usage (CPU, memory, etc.)	<code>docker stats</code>
<code>docker container cp</code>	Copy files between host and container	<code>docker cp file.txt myapp:/app/</code>
<code>docker container rename</code>	Rename a container	<code>docker rename old new</code>
<code>docker container commit</code>	Create new image from a container's changes	<code>docker commit myapp myimage:v2</code>

Container modes



■ Attached Mode

- The container runs in the foreground of your terminal, and you can see all its output (stdout/stderr) in real-time
- Pros
 - Terminal is "attached" to container's process
 - See all logs/output in real-time
 - Can send input to container (stdin)
- Cons
 - Terminal is blocked until container stops
 - Ctrl+C stops/kills the container
 - Closing terminal stops the container
- Use when
 - Debugging applications
 - Interactive sessions (shell, REPL)
 - Need to see immediate output
 - Development and testing

■ Detached Mode

- The container runs in the background, and the terminal returns immediately with the container ID.
- Pros
 - Terminal returns immediately
 - Container continues running after terminal closes
 - Can work on other tasks in same terminal
 - Ctrl+C doesn't affect the container
- Cons
 - Can't see logs without explicit command
 - Need to manually stop/kill the container
- Use when
 - Production services (web servers, databases)
 - Long-running background tasks
 - Microservices in a cluster
 - CI/CD pipelines
 - Services that run continuously

Publishing port on container



- **Port publishing** (also called **port mapping** or **port forwarding**) is the process of mapping a port on your **host machine** to a port inside a **Docker container**, allowing external traffic to reach services running inside the container
- **Core problem**
 - Docker containers run in **isolated networks** by default.
 - Each container gets its own network namespace with its own IP address and port space.
 - This isolation is great for security but creates a fundamental problem: **Services inside a container cannot be accessed from the outside world (including the host machine) without explicit port publishing**



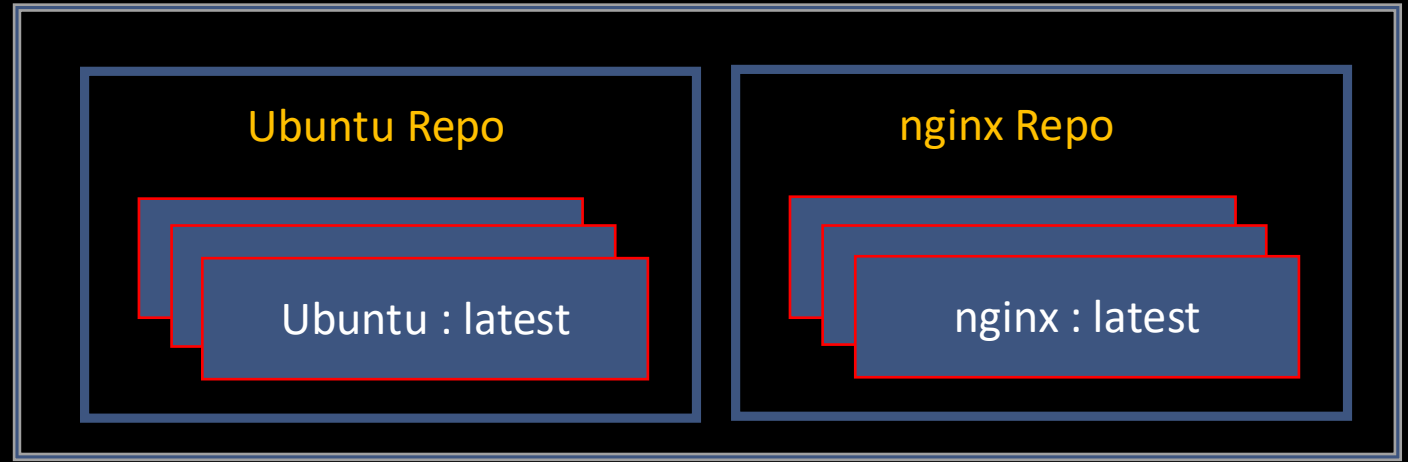
Docker Images (Advanced)

Docker Image



- Read-only instructions to run the containers
- It is made up of different layers
- Repositories hold images
- Docker registry stores repositories
- To create a custom image
 - Commit the running container
 - Use a Dockerfile
- Task
 - Create a container
 - Create a directory and a file within it
 - Commit the container to create a new image

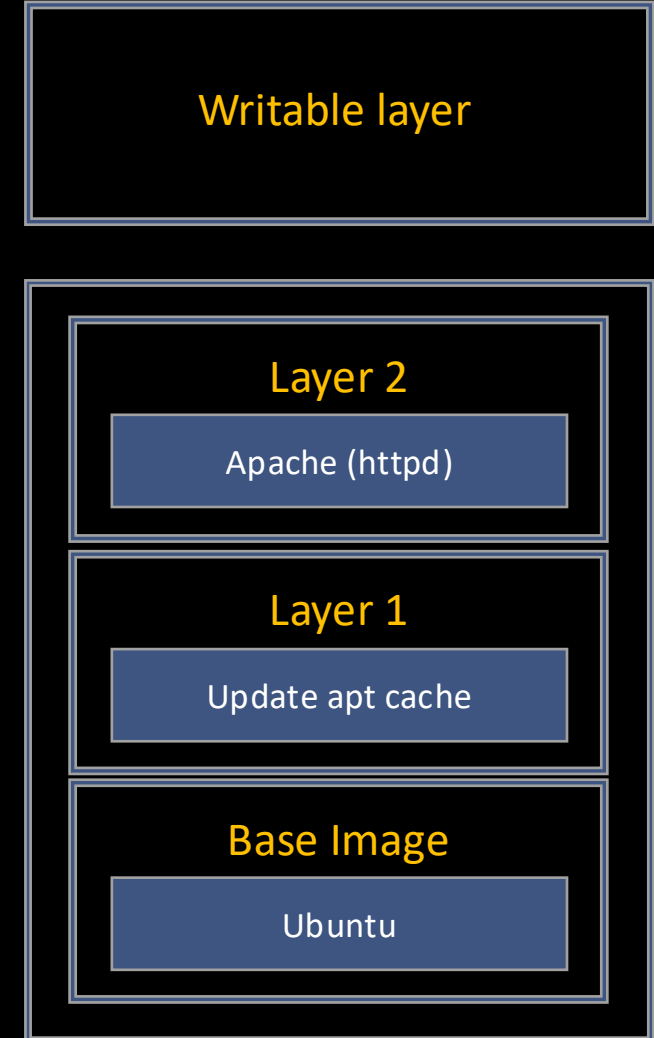
Docker Registry Server



Layered File System



- Docker images are made of layered FS
- Docker uses UnionFS for implementing the layered docker images
- Any update on the image adds a new layer
- All changes made to the running container are written inside a writable layer



Dockerfile



- The Dockerfile contains a series of instructions paired with arguments
- Each instruction should be in upper-case and be followed by an argument
- Instructions are processed from top to bottom
- Each instruction adds a new layer to the image and then commits the image
- Upon running, changes made by an instruction make it to the container

Dockerfile instructions



- FROM
- ENV
- RUN
- CMD
- EXPOSE
- WORKDIR
- ADD
- COPY
- LABEL
- MAINTAINER
- ENTRYPOINT



Orchestration

Container Orchestration



- Container orchestration is all about managing the lifecycles of containers, especially in large, dynamic environments
- Software teams use container orchestration to control and automate many tasks
 - Provisioning and deployment of containers
 - Redundancy and availability of containers
 - Scaling up or removing containers to spread application load evenly across host infrastructure
 - Movement of containers from one host to another if there is a shortage of resources in a host, or if a host dies
 - Allocation of resources between containers
 - External exposure of services running in a container with the outside world
 - Load balancing of service discovery between containers
 - Health monitoring of containers and hosts
 - Configuration of an application in relation to the containers running it
- Orchestration Tools
 - Docker Swarm
 - Kubernetes
 - Mesos
 - Marathon



Docker Swarm

- Docker Swarm is a container orchestration engine
- It takes multiple Docker Engines running on different hosts and lets you use them together
- The usage is simple: declare your applications as stacks of services, and let Docker handle the rest
- It is secure by default
- It is built using Swarmkit

What is a swarm?



- A swarm consists of multiple Docker hosts which run in **swarm mode**
- A given Docker host can be a manager, a worker, or perform both roles
- When you create a service, you define its optimal state
- Docker works to maintain that desired state
 - For instance, if a worker node becomes unavailable, Docker schedules that node's tasks on other nodes
- A *task* is a running container which is part of a swarm service and managed by a swarm manager, as opposed to a standalone container
- When Docker is running in swarm mode, you can still run standalone containers on any of the Docker hosts participating in the swarm, as well as swarm services
- A key difference between standalone containers and swarm services is that only swarm managers can manage a swarm, while standalone containers can be started on any daemon

Features



- Cluster management integrated with Docker Engine
- Decentralized design
- Declarative service model
- Scaling
- Desired state reconciliation
- Multi-host networking
- Service discovery
- Load balancing
- Secure by default
- Rolling updates

Nodes



- A **node** is an instance of the Docker engine participating in the swarm
- You can run one or more nodes on a single physical computer or cloud server
- To deploy your application to a swarm, you submit a service definition to a **manager node**
- **Manager Node**
 - The manager node dispatches units of work called tasks to worker nodes
 - Manager nodes also perform the orchestration and cluster management functions required to maintain the desired state of the swarm
 - Manager nodes elect a single leader to conduct orchestration tasks
- **Worker nodes**
 - Worker nodes receive and execute tasks dispatched from manager nodes
 - An agent runs on each worker node and reports on the tasks assigned to it
 - The worker node notifies the manager node of the current state of its assigned tasks so that the manager can maintain the desired state of each worker

Services and tasks

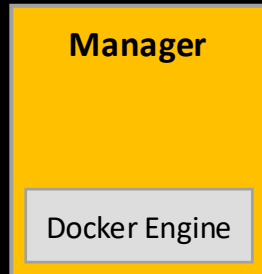


- **Service**
 - A service is the definition of the tasks to execute on the manager or worker nodes
 - It is the central structure of the swarm system and the primary root of user interaction with the swarm
 - When you create a service, you specify which container image to use and which commands to execute inside running containers
- **Task**
 - A task carries a Docker container and the commands to run inside the container
 - It is the atomic scheduling unit of swarm
 - Manager nodes assign tasks to worker nodes according to the number of replicas set in the service scale
 - Once a task is assigned to a node, it cannot move to another node
 - It can only run on the assigned node or fail

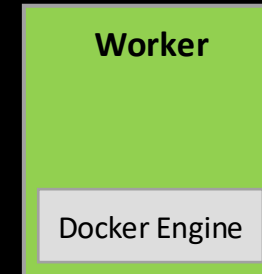


Create a Swarm and add workers to swarm

- In our swarm we are going to use 3 nodes (one manager and two workers)
- Every node must have Docker Engine 1.12 or newer installed
- Following ports are used in node communication
 - TCP port 2377 is used for cluster management communication
 - TCP and UDP port 7946 is used for node communication
 - UDP port 4789 is used for overlay network traffic



`docker swarm init --advertise-addr <ip>`



`docker swarm join --token <token>`

Swarm Setup



- Create swarm

- > `docker swarm init --advertise-addr <MANAGER-IP>`

- Get current status of swarm

- > `docker info`

- Get the list of nodes

- > `docker node ls`

Swarm Setup



- Get token (on manager node)
 - > `docker swarm join-token worker`
- Add node (on worker node)
 - > `docker swarm join --token <token>`

Overlay Network



- It is a computer network built on top of another network
- Sits on top of the host-specific networks and allows container, connected to it, to communicate securely
- When you initialize a swarm or join a host to swarm, two networks are created
 - An overlay network called as ingress network
 - A bridge network called as docker_gwbridge
- Ingress network facilitates load balancing among services nodes
- Docker_gwbridge is a bridge network that connect overlay networks to individual docker daemon's physical network

Service



- Definition of tasks to execute on Manager or Worker nodes
- Declarative Model for Services
- Scaling
- Desired state reconciliation
- Service discovery
- Rolling updates
- Load balancing
- Internal DNS component

Swarm Service



- **Deploy a service**

- > **docker service create --replicas <no> --name <name> -p <ports> <image> <command>**

- **Get running services**

- > **docker service ls**

- **Inspect service**

- > **docker service inspect <service>**

- **Get the nodes running service**

- > **docker service ps <service>**

Swarm Service



- **Scale service**

- > **docker service scale** <service>=<scale>

- **Update service**

- > **docker service update --image** <image> <service>

- **Delete service**

- > **docker service rm** <service>